

JavaChatApp

Documentação Técnica

Chat multiusuário em Java com servidor concorrente, cliente JavaFX e persistência SQLite

Vitor Miranda Jeremias

Estudante de Engenharia de Software - PUCPR

github.com/Vitormjere

1. Visão geral

Construí esse projeto como meu terceiro projeto de portfólio, depois de duas APIs web em C#/.NET e um pipeline de dados em Python. Dessa vez quis explorar um paradigma diferente: programação de rede concorrente, em vez de web APIs ou scripts sequenciais

O sistema é um chat multiusuário composto por um servidor Java que aceita várias conexões simultâneas e um cliente JavaFX que se conecta a esse servidor. Cada mensagem enviada por um usuário é retransmitida em tempo real para todos os outros usuários conectados, e o histórico de mensagens é salvo em um banco SQLite

Stack utilizada

- Servidor: Java 21, sockets TCP, multithreading
- Protocolo: JSON via Gson
- Persistência: SQLite, acessado via JDBC puro (sem ORM)
- Cliente: JavaFX
- Build: Maven
- Testes: JUnit 5

2. Arquitetura

O servidor fica em loop constante aceitando novas conexões. Para cada cliente que conecta, crio uma thread dedicada, responsável por ler as mensagens daquele cliente específico e devolver o controle para o servidor imediatamente, sem bloquear a aceitação de outros clientes

Mantenho uma lista compartilhada com todos os clientes conectados no momento. Quando uma mensagem chega em qualquer thread, percorro essa lista e envio a mensagem para todos os outros clientes, exceto para quem a enviou

As mensagens trafegam entre cliente e servidor no formato JSON, usando a biblioteca Gson. Defini dois tipos de mensagem: JOIN, usado quando um usuário entra no chat, e CHAT, usado para mensagens normais. Cada mensagem carrega o nickname do autor e um timestamp, usado tanto para exibição quanto para persistência

O cliente é construído em JavaFX, com uma thread separada dedicada a escutar mensagens vindas do servidor, enquanto a thread principal da interface cuida da digitação do usuário

3. Decisões técnicas e por quê

Uma thread por cliente conectado

Escolha: cada conexão de cliente é atendida por uma thread própria, criada assim que o servidor aceita a conexão

Motivação: esse modelo é o mais didático para aprender os fundamentos de concorrência. Cada conexão tem sua própria linha de execução isolada, e o raciocínio sobre o que acontece "ao mesmo tempo" fica mais direto de acompanhar do que com um pool de threads compartilhado

CopyOnWriteArrayList para a lista de clientes conectados

Escolha: a lista compartilhada de clientes ativos usa a estrutura CopyOnWriteArrayList

Motivação: a lista é lida com muito mais frequência (a cada broadcast) do que escrita (só quando alguém entra ou sai do chat), que é exatamente o cenário para o qual essa estrutura foi pensada. Ela permitiu evitar blocos synchronized manuais mantendo a lista segura para acesso concorrente

Protocolo JSON em vez de texto delimitado por caractere

Escolha: as mensagens trocadas entre cliente e servidor são serializadas em JSON, usando Gson

Motivação: considerei um formato de texto simples separado por um caractere especial, mas descartei essa opção porque ela quebra se o conteúdo da mensagem contiver esse mesmo caractere. JSON resolve esse problema automaticamente e é um formato já conhecido de APIs REST

JDBC puro, sem ORM, para a persistência

Escolha: acesso ao SQLite feito diretamente com SQL, via JDBC, sem um ORM

Motivação: decisão deliberada, para praticar uma camada de acesso a dados mais próxima do banco e reforçar o entendimento do JDBC nativo do Java, diferente da abordagem com Entity Framework usada nos projetos em C#

Java 21 LTS em vez da versão mais recente disponível

Escolha: o projeto foi configurado para rodar sobre o JDK 21 (LTS), embora a versão mais nova já estivesse instalada na máquina no início do projeto

Motivação: bibliotecas como JavaFX e o driver do SQLite têm suporte mais testado em versões LTS, e é também o padrão mais comum exigido em vagas de estágio e júnior

Classe Launcher separada da classe principal do JavaFX

Escolha: criação de uma classe Launcher, cujo único papel é chamar o método main da aplicação JavaFX

Motivação: o Java exige que aplicações JavaFX rodem através do sistema de módulos quando executadas fora de um ambiente configurado especificamente para isso, o que gera um erro de "JavaFX runtime components are missing" ao rodar a classe da aplicação diretamente. A classe Launcher contorna essa checagem

4. Desafios reais e como resolvi

Esta seção documenta problemas reais enfrentados durante o desenvolvimento, úteis para revisão antes de entrevistas técnicas

Dependências do Maven não sendo encontradas

Problema: depois de adicionar o JavaFX e o driver do SQLite no pom.xml, o IntelliJ reportava as duas dependências como não encontradas, mesmo com a internet funcionando normalmente (testado acessando o Maven Central direto pelo navegador)

Solução: identificado que era um cache de tentativa de download anterior que tinha falhado e ficado registrado como definitivo. Resolvido forçando um reload completo do projeto Maven pela IDE

Declaração de package no lugar errado

Problema: ao reescrever o Main.java, a linha package foi colocada depois dos imports por engano, gerando um erro de compilação genérico ("class, interface, enum, or record expected")

Solução: movida a linha package para a primeira posição do arquivo, já que em Java ela precisa vir antes de qualquer import

Mensagens duplicadas no broadcast

Problema: ao testar o chat com dois clientes, cada mensagem enviada aparecia duas vezes para quem a mandou

Solução: identificado que o servidor estava retransmitindo a mensagem de volta para o próprio remetente. Ajustado o método de broadcast para pular o cliente que originou a mensagem

Rodar múltiplas instâncias do mesmo cliente durante os testes

Problema: por padrão, o IntelliJ encerra a execução anterior de uma configuração ao rodá-la de novo, o que impedia testar dois clientes de chat simultaneamente

Solução: habilitada a opção "Allow multiple instances" na configuração de execução de cada classe cliente

5. Como rodar localmente

```
git clone https://github.com/Vitormjere/JavaChatApp.git
cd JavaChatApp
```

Abro o projeto na IDE e deixo o Maven resolver as dependências.

Para iniciar o servidor, rodo a classe Main

Para iniciar um cliente, rodo a classe Launcher, não a classe ChatGUI diretamente. Rodo mais de uma instância para simular vários usuários conversando ao mesmo tempo

Na primeira conexão, sou solicitado a digitar um nickname. As mensagens enviadas são retransmitidas para todos os outros clientes conectados e salvas no arquivo chat_history.db, criado na raiz do projeto

Testes

```
mvn test
```

Cobre a serialização JSON das mensagens e a persistência no banco, usando um banco de dados de teste isolado